

w



SENATI



Apellidos y Nombres: Huamani Feria Claudio

ID: 001597435

Dirección Zonal/CFP: Arequipa/Arequipa/Puno

Carrera: Ingeniería de software con Inteligencia artificial

Semestre: 4to Semestre

Instructor: Jesús Romero Villanueva

Curso: seminario de complementación práctica I

TAREAS Y OPERACIONES COVERTURADAS

-TH1_Realiza operaciones con las Librerías Pandas y Numpy

- Estudia los fundamentos de vectores y matrices.
- Define la librería Pandas y Numpy.
- Manipula y analiza estructuras de datos.
- Lee archivos CSV con Numpy y Pandas.

-TH2_Estudia el uso de las Librerías Scikit-learn y Pytorch

- Define el concepto de Machine Learning
- Define la librería Scikit-Learn y Pytorch.
- Identifica principales aplicaciones.

-TH3_Estudia el uso de las librerías SciPy y Nltk

- Define que el procesamiento de lenguaje natural (NLP)
- Define la librería SciPy y Nltk.
- Identifica principales aplicaciones del NLP

-TH4_ Estudia el uso de las Librerías Tensorflow y Keras

- Define el concepto de Deep Learning
- Define la librería Tensorflow y Keras.
- Identifica principales aplicaciones

-TH5_ Realiza operaciones con las librerías Matplotlib y Seaborn

- Define los histogramas y la importancia de la visualización de datos.
- Define la librería Matplotlib y Seaborn.
- Crea ejemplos de aplicación

-TH6_ Crea programas con algoritmos de aprendizaje supervisado

- Describe los tipos de algoritmos del aprendizaje supervisado.
- Define la regresión lineal simple y múltiple.
- Implementa algoritmo de regresión lineal simple con Python.

-TH6_ Crea programas con algoritmos de no aprendizaje supervisado

- Describe los tipos de algoritmos del aprendizaje supervisado.
- Define la regresión lineal simple y múltiple.
- Implementa algoritmo K-Means con Python.

TH7 Crea programas con algoritmos de aprendizaje supervisado

- Describe los tipos de algoritmos del aprendizaje supervisado.
- Define la regresión lineal simple y múltiple.
- Implementa algoritmo de regresión lineal simple con Python.

TH8 Define la estructura y crea una red neuronal artificial

- Define la red neuronal artificial y su importancia en la IA.
- Describe la estructura de una red neuronal artificial.
- Identifica los tipos de redes neuronales artificiales.
- Crea una red neuronal con Tensorflow y Keras

TH9 Describe los principios de visión computacional y Machine Learning

- Define la importancia de la Visión Computacional en IA.
- Describe los fundamentos de la detección de objetos.
- Define la segmentación y reconocimiento de patrones.
- Identifica las principales aplicaciones.

TH10 Explica el uso de la IA

- Utiliza aprendizaje automático
- Diferencia entre aprendizaje supervisado y no supervisado
- Diferencia entre datos e información
- Diferencia entre clasificación y regresión

TH11 Utiliza recursividad en la programación

- Usa la recursividad
- Define y crea la estructura Arbol
- Usa Algoritmos pre-order traversal
- Usa Algoritmos post-order traversal

TH12 Construye algoritmo de árbol de decisiones

- Usa algoritmos de árboles de decisión
- Aplica entropía y ganancia de información
- Resuelve un problema con el algoritmo ID3

Ejercicio 1

```
package Ejercicios;

import weka.core.Instances;

public class Ejercicio1 {

    public static void main(String[] args) {
        // Analisis exploratorio iris weka
        try {
            Instances iris = DataSource.read("wine.arff");// CARGAMOS EL DATASET (LECTURA)
            int nInstances = iris.numInstances();// OBTENEMOS LA CANTIDAD DE REGISTROS
            int nAttributes = iris.numAttributes();// CANTIDAD DE COLUMNAS (CONTANDO CON EL TARGET)
            System.out.println("Cantidad de muestras: " + nInstances);
            System.out.println("Cantidad de atributos: " + (nAttributes - 1));// RESTAS EL TARGET
            System.out.println("---Atributos---");
            for (int i = 0; i < iris.numAttributes(); i++) {
                System.out.println(iris.attribute(i).name());
            }
            System.out.println("---Instancias---");
            for (int i = 0; i < iris.numInstances(); i++) {
                System.out.println(iris.instance(i));
            }
        } catch (Exception e) {
        }
    }
}
```

Ejercicio 2

```
package Ejercicios;

import weka.classifiers.lazy.IBk;

public class Ejercicio2 {

    public static void main(String[] args) {
        try {

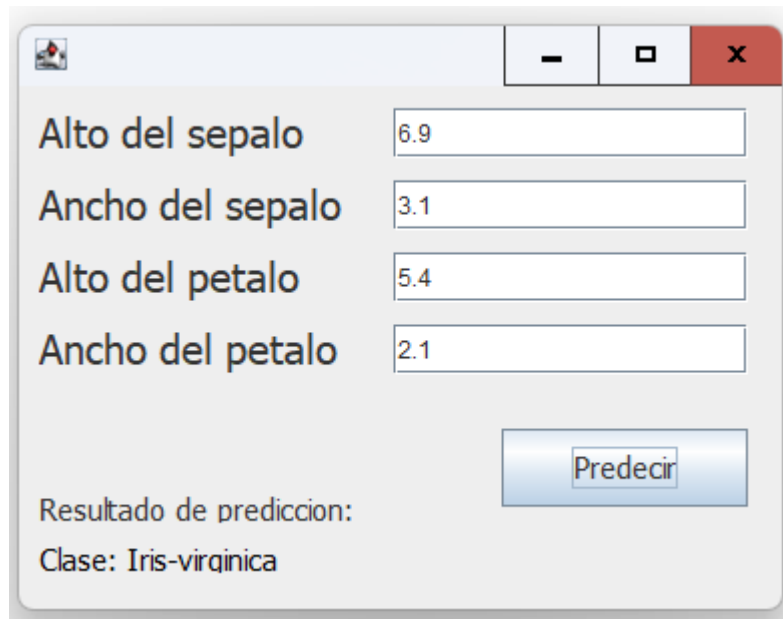
            //PROBAR CON OTROS REGISTRO (INGRESAR LOS DATOS POR TECLADO)
            Instances iris = DataSource.read("iris.arff");// CARGAMOS EL DATASET (LECTURA)
            int nInstances = iris.numInstances();// OBTENEMOS LA CANTIDAD DE REGISTROS
            int nAttributes = iris.numAttributes();// CANTIDAD DE COLUMNAS (CONTANDO CON EL TARGET)
            iris.setClassIndex(nAttributes-1); //ESTABLECER EL OBJETIVO (ULTIMA COLUMNA) TARGET
            IBk model = new IBk(); //DEFINIMOS NUESTRO MODELO (K VECINOS)
            model.setKNN(3);// ESTABLECEMOS NUESTRO HIPERPARAMETRO (3 VECINOS)
            model.buildClassifier(iris);//COMPILAMOS EL MODELO CONSTRUCTIVO

            //6.9,3.1,5.4,2.1,Iris-virginica
            //Definimos una muestra para la clasificación (Predicción)
            Instance florNueva= new DenseInstance(iris.numAttributes());
            florNueva.setValue(0, 6.9);
            florNueva.setValue(1, 3.1);
            florNueva.setValue(2, 5.4);
            florNueva.setValue(3, 2.1);
            florNueva.setDataset(iris);

            //Ejecutamos el modelo (predict)
            double index = model.classifyInstance(florNueva);
            String clase = iris.classAttribute().value((int) index);
            System.out.println(clase);
            System.out.println(index);

        } catch (Exception e) {
        }
    }
}
```

Tarea 1



Alto del sepalo 6.9

Ancho del sepalo 3.1

Alto del petalo 5.4

Ancho del petalo 2.1

Predecir

Resultado de prediccion:
Clase: Iris-virginica

Tarea 2

```
package Tarea;

import weka.classifiers.lazy.IBk;

public class Tarea1 {

    public static void main(String[] args) {
        try {
            Scanner scn = new Scanner(System.in);
            // PROBAR CON OTROS REGISTRO (INGRESAR LOS DATOS POR TECLADO)
            Instances iris = DataSource.read("iris.arff");// CARGAMOS EL DATASET (LECTURA)
            int nInstances = iris.numInstances();// OBTENEMOS LA CANTIDAD DE REGISTROS
            int nAttributes = iris.numAttributes();// CANTIDAD DE COLUMNAS (CONTANDO CON EL TARGET)
            iris.setClassIndex(nAttributes - 1); // ESTABLECER EL OBJETIVO (ULTIMA COLUMNA) TARGET
            IBk model = new IBk(); // DEFINIMOS NUESTRO MODELO (K VECINOS)
            model.setKNN(3);// ESTABLECEMOS NUESTRO HIPERPARAMETRO (3 VECINOS)
            model.buildClassifier(iris);// COMPILAMOS EL MODELO CONSTRUCTIVO

            // 6.9,3.1,5.4,2.1,Iris-virginica
            // Definimos una muestra para la clasificación (Predicción)
            Instance florNueva = new DenseInstance(iris.numAttributes());
            System.out.println("Ingrese el alto del sepalo");
            double sepal_alto = scn.nextDouble();
            System.out.println("Ingrese el ancho del sepalo");
            double sepal_ancho = scn.nextDouble();
            System.out.println("Ingrese el alto del petalo");
            double petal_alto = scn.nextDouble();
            System.out.println("Ingrese el ancho del petalo");
            double petal_ancho = scn.nextDouble();
            florNueva.setValue(0,sepal_alto);
            florNueva.setValue(1,sepal_ancho);
            florNueva.setValue(2, petal_alto);
            florNueva.setValue(3, petal_ancho);
            florNueva.setDataset(iris);

            // Ejecutamos el modelo (predict)
            double index = model.classifyInstance(florNueva);
            String clase = iris.classAttribute().value((int) index);
            System.out.println(clase);
            System.out.println(index);
        } catch (Exception e) {
            // TODO: handle exception
        }
    }
}
```

Tarea 3

```
package Tarea3;

import java.util.Scanner;

public class Tarea2 {

    public static void main(String[] args) {
        try {
            Scanner scn = new Scanner(System.in);
            Instances wine = DataSource.read("wine.arff");
            int nAttributes = wine.numAttributes();
            int nInstances = wine.numInstances();
            wine.setClassIndex(nAttributes-1);

            IBk model = new IBk();
            model.setKNN(3);
            model.buildClassifier(wine);

            Instance vinoNuevo = new DenseInstance(wine.numAttributes());
            System.out.println("Ingrese la cantidad de alcohol: ");
            double c_alco = scn.nextDouble();
            System.out.println("Ingrese la cantidad de acido malico: ");
            double c_acm = scn.nextDouble();
            System.out.println("Ingrese la cantidad de Ceniza: ");
            double c_cen = scn.nextDouble();
            System.out.println("Ingrese la cantidad de alcalinidad de las cenizas: ");
            int c_alce = scn.nextInt();
            System.out.println("Ingrese la cantidad de magnesio: ");
            int c_mg = scn.nextInt();
            System.out.println("Ingrese la cantidad de fenoles totales: ");
            double c_ft = scn.nextDouble();
            System.out.println("Ingrese la cantidad de flavanoides: ");
            double c fla = scn.nextDouble();
            System.out.println("Ingrese la cantidad de fenoles no flavanoides: ");
            double c_fnf = scn.nextDouble();
            System.out.println("Ingrese la cantidad de proantocianinas: ");
            double c_pro = scn.nextDouble();
            System.out.println("Ingrese la cantidad de Intensidad del color: ");
            double c_idc = scn.nextDouble();
            System.out.println("Ingrese la cantidad de Tono: ");
            double c_ton = scn.nextDouble();
            System.out.println("Ingrese el valor de D0280_D0315: ");
            double c_do2 = scn.nextDouble();
            System.out.println("Ingrese la cantidad de Prolina: ");
            int c_prol = scn.nextInt();
            vinoNuevo.setValue(0, c_alco);
            vinoNuevo.setValue(1, c_acm);
            vinoNuevo.setValue(2, c_cen);
            vinoNuevo.setValue(3, c_alce);
            vinoNuevo.setValue(4, c_mg);
            vinoNuevo.setValue(5, c_ft);
            vinoNuevo.setValue(6, c fla);
            vinoNuevo.setValue(7, c_fnf);
            vinoNuevo.setValue(8, c_pro);
            vinoNuevo.setValue(9, c_idc);
            vinoNuevo.setValue(10, c_ton);
            vinoNuevo.setValue(11, c_do2);
            vinoNuevo.setValue(12, c_prol);
            vinoNuevo.setDataset(wine);

            double index = model.classifyInstance(vinoNuevo);
            String clase = wine.classAttribute().value((int)index);
            System.out.println(clase);
            System.out.println(index);
        } catch (Exception e) {
            // TODO: handle exception
        }
    }
}
```

Ejercicio 1

```
package Ejercicios;

import java.io.File;

public class Ejercicio1 {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost/colegio";
        String user = "root";
        String password = "";
        try {
            //1 Enlace a la DB y ejecucion de la consulta
            Connection cn = DriverManager.getConnection(url,user,password);
            String sql = "select edad,asistencia,promedio,conducta,apoyo_familiar,abandono from estudiantes_riesgo";
            PreparedStatement ps = cn.prepareStatement(sql);
            ResultSet rs = ps.executeQuery(); //Los datos de la tabla ya se encuentran en el resulset
            System.out.println("CONECTADO");
            //2 Armandos el esqueleto del dataset
            ArrayList<Attribute> atributos = new ArrayList<Attribute>();
            //2.1 Definimos columnas
            atributos.add(new Attribute("edad"));
            atributos.add(new Attribute("asistencia"));
            atributos.add(new Attribute("promedio"));

            //NOTA: One_hot_encoder de conducta
            ArrayList<String> valoresConducta = new ArrayList<String>();
            valoresConducta.add("buena");
            valoresConducta.add("regular");
            valoresConducta.add("mala");
            atributos.add(new Attribute("conducta", valoresConducta));

            //NOTA: ONE_HOT_ENCODER DE APOYO_FAMILIAR
            ArrayList<String> valoresApoyo_f = new ArrayList<String>();
            valoresApoyo_f.add("si");
            valoresApoyo_f.add("no");
            atributos.add(new Attribute("apoyo_familiar", valoresApoyo_f));

            //NOTA: ONE_HOT_ENCODER DE ABANDONO
            ArrayList<String> valoresAbandono = new ArrayList<String>();
            valoresAbandono.add("si");
            valoresAbandono.add("no");
            atributos.add(new Attribute("abandono", valoresAbandono));

            //2.2 Declaramos el Dataset
            Instances data = new Instances("dataset",atributos,0);
            data.setClassIndex(data.numAttributes()-1);

            //3 Rellenamos el dataset (Del ResultSet Al Dataset)
            while (rs.next()) {
                Instance instance = new DenseInstance(data.numAttributes());
                instance.setValue(atributos.get(0), rs.getInt("edad"));
                instance.setValue(atributos.get(1), rs.getInt("asistencia"));
                instance.setValue(atributos.get(2), rs.getDouble("promedio"));
                instance.setValue(atributos.get(3), rs.getString("conducta"));
                instance.setValue(atributos.get(4), rs.getString("apoyo_familiar"));
                instance.setValue(atributos.get(5), rs.getString("abandono"));
                instance.setDataset(data);
                data.add(instance);
            }
            //3.1 ANÁLISIS EXPLORATORIO
            System.out.println(" --- DATASET GENERADO CORRECTAMENTE --- ");
            System.out.println("CANTIDAD DE COLUMNAS: "+data.numAttributes());
            System.out.println("CANTIDAD DE FILAS: "+data.numInstances());
            System.out.println(data.instance(0));
            for (int i = 0; i < data.numInstances(); i++) {
                System.out.println(data.instance(i).value(0));
            }

            //4 Implementacion del modelo
            Classifier model = new J48(); //Declaramos el modelo
            try {
                //4.1 Declaracion y construccion del modelo
                model.buildClassifier(data); //Compilacion y construccion del modelo de clasificacion
                //4.2 muestra para la prediccion
                Instance muestra = new DenseInstance(data.numAttributes());
                muestra.setValue(atributos.get(0), 17); //edad
                muestra.setValue(atributos.get(1), 90); //asistencia
                muestra.setValue(atributos.get(2), 19.0); //promedio
                muestra.setValue(atributos.get(3), "regular"); //conducta
                muestra.setValue(atributos.get(4), "si"); //apoyo_familiar
                muestra.setDataset(data);
                //4.3 Prediccion del modelo
                double indice = model.classifyInstance(muestra);
                String clase = data.classAttribute().value((int)indice);

                System.out.println("Prediccion de abandono: "+ clase);
                System.out.println(" --- Construcción del arbol --- ");
                System.out.println(model);
                System.out.println("Generación del archivo ARFF");
                ArffSaver arff=new ArffSaver();
                arff.setInstances(data);
                arff.setFile(new File("data.arff"));
                arff.writeBatch();
            } catch (Exception e) {
                e.printStackTrace();
            }
        } catch (SQLException e) {
            System.out.println("ERROR EN LA CONEXION");
            e.printStackTrace();
        }
    }
}
```


Tarea 1

edad	asiste...	prome...	condu...	apoyo...	aband...
16.0	85.0	14.2	0.0	0.0	1.0
17.0	60.0	11.5	1.0	1.0	0.0
18.0	92.0	16.8	0.0	0.0	1.0
17.0	45.0	10.0	2.0	1.0	0.0
16.0	70.0	12.6	1.0	0.0	1.0
19.0	30.0	9.8	2.0	1.0	0.0
18.0	78.0	13.9	0.0	0.0	1.0
17.0	55.0	10.2	2.0	1.0	0.0

Edad

Asistencia

Promedio

Conducta

Apoyo Familiar

Prediccion de abandono: no

predecir

Tarea 2

```

Arbol.java
7
8 public class Arbol {
9     public static void main(String[] args) {
10         try {
11             new DSUtil().crearDataset();
12             Instances data = DataSource.read("data.arff");
13
14             // Establecer la variable objetivo
15             data.setClassIndex(data.numAttributes() - 1);
16
17             // Crear y entrenar el modelo J48
18             J48 model = new J48();
19             model.buildClassifier(data);
20
21             System.out.println(model);
22
23             // Realizar una prediccion
24             Instance nuevo = data.firstInstance();
25             double prediccion = model.classifyInstance(nuevo);
26             String clase = data.classAttribute().value((int) prediccion);
27
28             System.out.println("Prediccion para el primer registro: " + clase);
29         } catch (Exception e) {
30
31         }
32     }
33 }

DAO.java
1 package Ejercicios;
2
3 import java.sql.Connection;
4
5 public class DAO {
6
7     public ResultSet obtenerDatos() throws SQLException {
8         String url="jdbc:mysql://localhost/colegio";
9         String user="root";
10        String password="";
11        Connection cn= DriverManager.getConnection(url, user, password);
12        String consulta="select edad, asistencia, promedio, "
13        + "conducta, apoyo_familiar, abandono "
14        + "from estudiantes_riesgo";
15        PreparedStatement ps=cn.prepareStatement(consulta);
16        ResultSet rs=ps.executeQuery();
17        return rs;
18    }
19 }

DSUtil.java
14 public class DSUtil {
15
16     public void crearDataset() throws Exception {
17         // Obtener datos
18         ResultSet rs = new DAO().obtenerDatos();
19
20         // Definir atributos
21         ArrayList<Attribute> atributos = new ArrayList<>();
22         atributos.add(new Attribute("edad"));
23         atributos.add(new Attribute("asistencia"));
24         atributos.add(new Attribute("promedio"));
25
26         ArrayList<String> valoresConducta = new ArrayList<>();
27         valoresConducta.add("buena");
28         valoresConducta.add("regular");
29         valoresConducta.add("mala");
30         atributos.add(new Attribute("conducta", valoresConducta));
31
32         ArrayList<String> valoresApoyo = new ArrayList<>();
33         valoresApoyo.add("si");
34         valoresApoyo.add("no");
35         atributos.add(new Attribute("apoyo_familiar", valoresApoyo));
36
37         ArrayList<String> valoresAbandono = new ArrayList<>();
38         valoresAbandono.add("si");
39         valoresAbandono.add("no");
40         atributos.add(new Attribute("abandono", valoresAbandono));
41
42         // Crear dataset
43         Instances data = new Instances("dataset", atributos, 0);
44         data.setClassIndex(data.numAttributes() - 1);
45
46         // Llenar dataset con los datos del ResultSet
47         while (rs.next()) {
48             Instance instance = new DenseInstance(data.numAttributes());
49             instance.setValue(atributos.get(0), rs.getInt("edad"));
50             instance.setValue(atributos.get(1), rs.getInt("asistencia"));
51             instance.setValue(atributos.get(2), rs.getDouble("promedio"));
52             instance.setValue(atributos.get(3), rs.getString("conducta"));
53             instance.setValue(atributos.get(4), rs.getString("apoyo_familiar"));
54             instance.setValue(atributos.get(5), rs.getString("abandono"));
55             instance.setDataset(data);
56             data.add(instance);
57         }
58
59         // Guardar en ARFF
60         ArffSaver arff = new ArffSaver();
61         arff.setInstances(data);
62         arff.setFile(new File("data.arff"));
63         arff.writeBatch();
64
65         rs.close();
66     }
67 }

```